Project Topic Catalog

Applied Parallel Programming

This document lists all pre-approved project topics for Part 2 of the course. Read it carefully before your Week 6 session. By the end of that session, your team must have claimed a topic, submitted a proposal, and pushed a running CPU baseline to your Git repository.

How to Use This Catalog

This catalog is a reference, not a constraint. You are strongly encouraged to bring your own topic. If you have a problem you care about — from your research, your job, or your own curiosity — that is the best possible starting point. Use the entries below as examples of what a well-scoped project looks like: the level of detail expected in the proposal, the structure of an optimization plan, and what a "measurable performance target" means in practice. See the Custom Topic section for guidelines on proposing one.

Each topic entry contains:

- **What it is** — a short description of the problem and why it benefits from GPU acceleration
- **What you implement** — the specific pipeline your team is responsible for building and optimizing
- **Performance target** — a concrete, measurable speedup goal
- **Suggested dataset** — where to get data and how to load it
- **Reading list** — papers, tutorials, and reference code to get you started (read before Week 7, not after)

The Partial GPU Principle

You do not need to move everything to GPU. In fact, doing so is usually the wrong approach. Real-world GPU programming is about identifying the one or two operations that dominate runtime and accelerating those — not rewriting an entire application.

Your workflow for every project is:

1. **Build a complete CPU pipeline first.** Use whatever Python libraries are appropriate: PyTorch, scikit-learn, OpenCV, NetworkX. The CPU pipeline is your correctness reference and your timing baseline.

2. **Profile it.** Run `cProfile` or `torch.profiler` and find the top functions by cumulative time. These are your GPU targets.
3. **Justify your choice in the proposal.** Write one paragraph explaining why you chose that specific operation: what fraction of total runtime it takes, why it is parallel, and what GPU technique you will apply.
4. **Replace only that part with a custom CUDA kernel.** Everything else stays as-is. The rest of the pipeline calls your kernel the same way it would call any library function.
5. **Verify end-to-end correctness.** After the swap, the full pipeline must produce identical results to the CPU reference within a specified tolerance.

This means a project built around a PyTorch model is completely valid. The model's forward and backward passes stay in PyTorch. You write a kernel for the bottleneck operation, drop it in, and measure the improvement. The same principle applies across all tracks: use libraries for parts that are already fast; write kernels only where profiling tells you to.

The bottleneck analysis section of your proposal is where you demonstrate that you have actually profiled the CPU pipeline and made a reasoned decision — not just guessed at what is slow.

Track A — Computer Vision

Computer vision problems are naturally data-parallel: the same operation is applied independently to every pixel, every patch, or every image in a batch. This makes them excellent GPU targets even for students with limited parallel programming experience.

A1 · HOG + Linear SVM Image Classification

Problem: Image classification on CIFAR-10 using handcrafted features and a classical classifier.

What it is: Before deep learning, the standard computer vision pipeline was: extract handcrafted features from images → train a classifier on those features. Histogram of Oriented Gradients (HOG) captures the distribution of gradient orientations in local image patches. A Linear SVM then classifies images based on those feature vectors. The bottleneck is HOG extraction: computing gradients, binning orientations into histograms, and normalizing blocks — all of which must be done for every image, every cell, every pixel. This is a textbook map-reduce problem and parallelizes extremely well.

What you implement:

1. **CPU pipeline:** Full HOG + SVM pipeline using scikit-image and scikit-learn. Profile with `cProfile` to confirm HOG extraction dominates runtime (it will — SVM inference on precomputed features is fast).

2. **Bottleneck:** HOG feature extraction — gradient computation, orientation binning, block normalization. This is your GPU target. SVM training and inference stay in scikit-learn.
3. **GPU V1:** Naive Numba kernel for per-pixel gradient computation (magnitude + orientation).
4. **GPU V2:** Shared memory tiling for block normalization; coalesced histogram accumulation with atomics.
5. **GPU V3:** Fused kernel combining gradient → histogram → normalization in a single pass; CUDA streams for batch processing.
6. **Benchmark:** HOG extraction throughput (images/second) at each version; end-to-end pipeline accuracy must remain unchanged.

Performance target: Process the full CIFAR-10 test set (10,000 images) in under 1 second. Target 50–100× speedup over the CPU HOG extraction step.

Dataset: CIFAR-10 — 60,000 32×32 RGB images, 10 classes.

- Download: https://www.cs.toronto.edu/~kriz/cifar.html
- Python loader: `pip install tensorflow-datasets` or load raw binary with NumPy

Reading list:

- Dalal, N. & Triggs, B. (2005). *Histograms of Oriented Gradients for Human Detection*. CVPR 2005. — the original HOG paper. Read Sections 1–3. https://lear.inrialpes.fr/people/triggs/pubs/Dalal-cvpr05.pdf
- scikit-image HOG docs and source: https://scikit-image.org/docs/stable/auto_examples/features_detection/plot_hog.html
- Hsu, C., Chang, C., Lin, C. (2003). *A Practical Guide to Support Vector Classification*. — short, practical, covers the math without being overwhelming. https://www.csie.ntu.edu.tw/~cjlin/papers/guide/guide.pdf
- OpenCV HOG source (for implementation reference, not to copy): https://github.com/opencv/opencv/blob/master/modules/objdetect/src/hog.cpp

---

A2 · Dense Optical Flow Estimation

Problem: Compute per-pixel motion vectors between consecutive video frames.

What it is: Optical flow answers the question: for each pixel in frame t, where did it come from in frame t−1? The Lucas-Kanade method solves a local least-squares system at every pixel, using image gradients and temporal differences as inputs. The algorithm is inherently iterative (coarse-to-fine pyramid) and involves solving thousands of small independent 2×2 linear systems simultaneously — a perfect fit for GPU parallelism. Dense optical flow is a foundational algorithm in video analysis, autonomous driving, and medical imaging.

What you implement:

1. **CPU pipeline:** Full Lucas-Kanade optical flow pipeline using OpenCV (`cv2.calcOpticalFlowFarneback` for verification; your own NumPy LK for baseline timing). Profile to confirm the pyramid solve dominates — not I/O.
2. **Bottleneck:** The dense per-pixel least-squares solve at each pyramid level. Gradient computation and bilinear interpolation are secondary targets. Everything else (image loading, visualisation) stays in Python/OpenCV.
3. **GPU V1:** Parallel gradient computation kernel; naive per-pixel warp at each pyramid level.
4. **GPU V2:** Shared memory for patch-level data reuse; minimize redundant global reads across neighbouring pixels.
5. **GPU V3:** Kernel fusion across pyramid levels; texture memory for bilinear interpolation.
6. **Benchmark:** Latency per frame pair at 480p and 1080p; verify endpoint error (EPE) against OpenCV reference.

Performance target: Process a 480p frame pair in under 20 ms (real-time threshold). Target 40–100× speedup over the CPU pyramid solve.

Dataset: Middlebury Optical Flow dataset (ground truth flow available for correctness verification).

- Download: https://vision.middlebury.edu/flow/data/
- Or use any consecutive video frames; ground truth is only needed for the correctness check

Reading list:

- Lucas, B. & Kanade, T. (1981). *An Iterative Image Registration Technique with an Application to Stereo Vision*. IJCAI 1981. — the foundational paper. Only 6 pages. https://ri.cmu.edu/pub_files/pub3/lucas_bruce_d_1981_2/lucas_bruce_d_1981_2.pdf
- Bouguet, J. (2001). *Pyramidal Implementation of the Lucas Kanade Feature Tracker*. Intel Tech Report. — the practical pyramid variant you will implement. https://robots.stanford.edu/cs223b04/algo_tracking.pdf
- OpenCV optical flow tutorial: https://docs.opencv.org/4.x/d4/dee/tutorial_optical_flow.html
- GPU optical flow survey: Zach, C. et al. (2007). *A Duality Based Approach for Realtime TV-L1 Optical Flow*. DAGM 2007.

A3 · Stereo Depth Estimation

Problem: Compute a dense depth map from a calibrated stereo image pair.

What it is: A stereo camera provides two images of the same scene from slightly different viewpoints. For every pixel in the left image, the algorithm searches the right image along the same horizontal scanline to find the best match — this offset (disparity) is inversely proportional

to depth. The naive approach (Semi-Global Matching, or SGM) builds a 3D cost volume of size H×W×D where D is the maximum disparity range, then aggregates costs along multiple paths. The cost volume is the bottleneck: it is large, the aggregation is partially sequential, and the memory access patterns are irregular. This topic gives students hands-on experience with irregular parallelism and the tension between sequential dynamic programming and GPU throughput.

What you implement:

1. **CPU pipeline:** Full stereo pipeline using OpenCV (`cv2.StereoBM`) for end-to-end verification; your own NumPy SAD block-matching for baseline timing. Profile to confirm cost volume construction dominates.
2. **Bottleneck:** Cost volume construction — computing matching costs for every (pixel, disparity) pair. Winner-Takes-All selection is a secondary target. Image loading and rectification stay in OpenCV.
3. **GPU V1:** Parallel cost volume construction — one thread per (pixel, disparity) pair.
4. **GPU V2:** Shared memory for block-matching patches; coalesced disparity volume writes.
5. **GPU V3:** Fused kernel combining cost computation + WTA disparity selection + subpixel refinement.
6. **Benchmark:** Latency for disparity map at 640×480 and 1280×720; verify bad-pixel rate against OpenCV reference.

Performance target: Produce a disparity map at 640×480 in under 50 ms. Target 50–150× speedup over the CPU cost volume step.

Dataset: KITTI Stereo 2015 or Middlebury Stereo.

- KITTI: https://www.cvlibs.net/datasets/kitti/eval_stereo.php
- Middlebury: https://vision.middlebury.edu/stereo/data/

Reading list:

- Hirschmüller, H. (2008). *Stereo Processing by Semiglobal Matching and Mutual Information*. IEEE TPAMI. — the SGM paper that defines the algorithm you will implement. https://core.ac.uk/download/pdf/11134866.pdf
- Hammerstein, F. et al. (2021). *Efficient Semi-Global Matching for Real-Time Applications*. — covers GPU-specific implementation strategies.
- OpenCV stereo tutorial: https://docs.opencv.org/4.x/dd/d53/tutorial_py_depthmap.html
- KITTI stereo benchmark leaderboard (for context): https://www.cvlibs.net/datasets/kitti/eval_stereo.php

---

A4 · Real-Time Non-Maximum Suppression for Object Detection

Problem: Accelerate the post-processing stage of object detectors — specifically, the bounding box filtering step (NMS).

What it is: Modern object detectors (YOLO, SSD, Faster R-CNN) output thousands of candidate bounding boxes per image. Non-Maximum Suppression (NMS) filters these down to the final detections by suppressing boxes that overlap heavily with higher-confidence boxes. On CPU, NMS is a serial $O(n^2)$ operation that becomes the throughput bottleneck at inference time. Parallel NMS on GPU can process all box pairs simultaneously. This topic is practical and industry-relevant — NMS acceleration is used in production inference pipelines at every major tech company.

What you implement:

1. **CPU pipeline:** Full object detection inference using a pretrained YOLO or SSD model in PyTorch, followed by greedy NMS in NumPy. Profile to confirm NMS is the post-processing bottleneck at large batch sizes.
2. **Bottleneck:** NMS — IoU matrix computation + iterative suppression. The model forward pass stays in PyTorch/cuDNN.
3. **GPU V1:** Parallel IoU matrix kernel — one thread per (box_i, box_j) pair.
4. **GPU V2:** Batched NMS using parallel reduction to build the suppression mask; coalesced box coordinate reads.
5. **GPU V3:** Matrix NMS (Wang et al. 2020) — replace serial greedy NMS with a fully parallel soft-suppression pass.
6. **Benchmark:** NMS latency for $N \in \{100, 1000, 10{,}000\}$ boxes; verify final detections match torchvision NMS within 1e-4.

Performance target: Process 10,000 boxes at batch size 32 in under 5 ms. Target 30–80× speedup over the CPU NMS step.

Dataset: No special dataset needed. Generate synthetic bounding boxes, or use COCO detection outputs from a pre-trained detector.

- COCO: https://cocodataset.org
- Or use `torchvision.ops.box_iou` on CPU for ground-truth verification

Reading list:

- Bodla, N. et al. (2017). *Soft-NMS — Improving Object Detection With One Line of Code*. ICCV 2017. https://arxiv.org/abs/1704.04503
- Wang, X. et al. (2020). *SOLOv2: Dynamic and Fast Instance Segmentation*. NeurIPS 2020. — introduces Matrix NMS (Section 3.3). https://arxiv.org/abs/2003.10152
- Hosang, J. et al. (2017). *Learning Non-Maximum Suppression*. CVPR 2017. https://arxiv.org/abs/1705.02950
- NVIDIA TensorRT NMS plugin source (reference implementation): https://github.com/NVIDIA/TensorRT

A5 · Image Processing Pipeline

Problem: Build a multi-stage image processing pipeline (filter → edge detection → histogram equalization) and maximize end-to-end GPU throughput using streams and kernel fusion.

What it is: A classic multi-stage pipeline where each stage transforms the image and passes it to the next. Individually, each stage is simple and well-understood. The challenge is the *pipeline* itself: how do you keep the GPU busy when stages have different compute intensities? How do you avoid the overhead of multiple kernel launches and intermediate global memory writes? This topic teaches pipelining, kernel fusion, and CUDA streams — skills that transfer directly to production inference pipelines.

What you implement:

1. **CPU pipeline:** Full OpenCV pipeline (Gaussian blur → Sobel edge detection → CLAHE histogram equalization) on a batch of high-resolution images. Profile to determine which stage costs the most — it is typically the blur or the CLAHE step.
2. **Bottleneck:** The one or two stages identified by your profiler. You do not need to GPU-accelerate all three; accelerating the dominant stage is sufficient for V1–V2.
3. **GPU V1:** Separate Numba kernels for your chosen stages; sequential execution.
4. **GPU V2:** Shared memory tiling in each kernel; coalesced reads and writes.
5. **GPU V3:** Fused kernel combining all three stages; eliminate intermediate global memory writes between stages.
6. **GPU V4 (optional):** CUDA streams to overlap kernel execution with host-to-device transfers for a batch of images.
7. **Benchmark:** Throughput in megapixels/second at 720p and 4K; verify pixel-level output matches OpenCV within tolerance.

Performance target: Process a 4K (3840×2160) image through the full pipeline in under 10 ms. Target 20–60× speedup over the CPU bottleneck stage.

Dataset: Any image dataset. Recommended: DIV2K high-resolution images.

- DIV2K: https://data.vision.ee.ethz.ch/cvl/DIV2K/
- Or use any set of high-resolution photos

Reading list:

- NVIDIA CUDA C Best Practices Guide, Chapter 9 (Memory Optimization): https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/
- Harris, M. (2007). *Optimizing Parallel Reduction in CUDA*. NVIDIA Technical Report. — foundational thinking for kernel fusion.
- Merrill, D. & Garland, M. (2016). *Single-pass Parallel Prefix Scan with Decoupled Look-back*. — relevant for the histogram stage.

- OpenCV GPU module source (reference): https://github.com/opencv/opencv_contrib/tree/master/modules/cudafilters

---

Track B — Machine Learning

ML workloads are the primary use case for modern GPU hardware. These topics give students experience with the algorithms that underpin deep learning frameworks, implemented from scratch rather than borrowed from PyTorch.

B1 · U-Net Inference Acceleration for Medical Image Segmentation

Problem: Accelerate the computationally intensive parts of U-Net inference for biomedical image segmentation, specifically the transposed convolution (upsampling) and skip-connection aggregation steps that dominate the decoder.

What it is: U-Net is the dominant architecture for medical image segmentation (tumor detection, cell boundary delineation, organ segmentation). Its encoder-decoder structure with skip connections means the decoder must repeatedly upsample feature maps and fuse them with high-resolution encoder outputs — a pattern that involves large intermediate tensors, many memory reads from different resolutions, and transposed convolutions that are poorly served by the generic cuDNN GEMM path. Profiling a standard PyTorch U-Net reveals that on large inputs (e.g., 512×512 with multiple channels), the decoder upsampling steps and skip-connection concatenations account for a disproportionate share of inference time and memory bandwidth. This topic gives students experience with a real production architecture, irregular memory access patterns in the decoder, and the trade-off between compute and memory bandwidth in encoder-decoder networks.

What you implement:

1. **CPU pipeline:** Full U-Net inference in PyTorch (CPU) on a batch of medical images. Use `torch.profiler` to measure per-layer time. Confirm which decoder layers dominate — typically the early high-resolution upsampling blocks.
2. **Bottleneck:** The transposed convolution + skip-connection concatenation in the decoder. The encoder and final classification head stay in PyTorch/cuDNN.
3. **GPU V1:** Custom Numba kernel for bilinear upsampling (replaces `nn.ConvTranspose2d` in the targeted decoder blocks); verify feature map values match PyTorch within 1e-5.
4. **GPU V2:** Fused kernel combining upsampling + skip-connection channel concatenation in a single pass; eliminates the intermediate tensor write between the two operations.
5. **GPU V3:** Shared memory tiling for the fused kernel; coalesced reads from both the upsampled and skip-connection feature maps; overlap decoder blocks with CUDA streams.

6. **Benchmark:** Decoder inference latency (targeted blocks only) and end-to-end inference latency for input sizes 256×256 and 512×512, batch size 4; segmentation IoU must remain unchanged after the swap.

Performance target: Reduce decoder latency of the targeted upsampling blocks by at least 3× vs PyTorch baseline on 512×512 inputs. End-to-end segmentation accuracy (mean IoU) must not degrade by more than 0.1%.

Important constraint: Encoder, batch normalisation, activations, and the final 1×1 classifier head stay in PyTorch. Only the upsampling and skip-connection fusion in the targeted decoder blocks are replaced.

Dataset: ISIC 2018 skin lesion segmentation dataset (binary masks, accessible without institutional approval).

- Download: https://challenge.isic-archive.com/data/#2018
- Or use Oxford-IIIT Pet dataset for quick iteration: `torchvision.datasets.OxfordIIITPet`
- Pretrained U-Net weights: `segmentation-models-pytorch` library (`pip install segmentation-models-pytorch`)

Reading list:

- Ronneberger, O. et al. (2015). *U-Net: Convolutional Networks for Biomedical Image Segmentation*. MICCAI 2015. — the original paper; read Section 2 for the architecture. https://arxiv.org/abs/1505.04597
- Yakubovskiy, P. (2019). *Segmentation Models PyTorch*. — practical pretrained U-Net implementations. https://github.com/qubvel/segmentation_models.pytorch
- NVIDIA Deep Learning Examples — U-Net Medical: https://github.com/NVIDIA/DeepLearningExamples/tree/master/PyTorch/Segmentation/nnUNet
- Shi, W. et al. (2016). *Real-Time Single Image and Video Super-Resolution Using an Efficient Sub-Pixel Convolutional Neural Network*. CVPR 2016. — introduces sub-pixel shuffling as an alternative to transposed conv; useful context for your V3 design. https://arxiv.org/abs/1609.05158

B2 · K-Means Clustering at Scale

Problem: Implement and optimize K-Means clustering for large datasets (N > 1M points, D > 100 dimensions).

What it is: K-Means alternates between two steps: assignment (find the nearest centroid for every point) and update (recompute each centroid as the mean of its assigned points). The assignment step requires computing N×K distances — a matrix operation that is embarrassingly

parallel. The update step is a reduction. For large N and D, the CPU implementation is too slow for practical use. GPU K-Means is a classic example of how a simple algorithm becomes highly non-trivial to optimize at scale: the distance matrix does not fit in L2 cache, centroid updates require atomic operations or multi-stage reduction, and load imbalance between clusters degrades performance.

What you implement:

1. **CPU pipeline:** Full K-Means using scikit-learn `KMeans` for verification; your own NumPy implementation for baseline timing. Profile to confirm the assignment step dominates (it always does for large N and K).
2. **Bottleneck:** The assignment step — computing N×K distances and finding the argmin per point. The update step (centroid recomputation) is a secondary target.
3. **GPU V1:** Naive assignment kernel — one thread per point, sequential loop over K centroids.
4. **GPU V2:** Shared memory to cache centroid blocks; coalesced distance accumulation across the D dimensions.
5. **GPU V3:** Reformulate distance as $||x-c||^2 = ||x||^2 - 2x \cdot c + ||c||^2$ and reduce to a CuBLAS matrix multiply for the cross term.
6. **GPU V4 (optional):** Yinyang K-Means — group centroids into superclusters to skip provably unnecessary distance computations.
7. **Benchmark:** Per-iteration time and convergence time for N=1M, K=256, D=128; verify final cluster assignments match scikit-learn within expected randomness tolerance.

Performance target: Complete 100 iterations for N=1M, K=256, D=128 in under 10 seconds. Target 20–80× speedup over the CPU assignment step.

Dataset:

- Synthetic: generate with `numpy.random.randn(N, D)` — reproducible, controllable
- Real: GloVe word embeddings (300D, 1.9M vectors) — download at https://nlp.stanford.edu/projects/glove/

Reading list:

- Lloyd, S. (1982). *Least Squares Quantization in PCM*. IEEE Transactions on Information Theory. — the original K-Means paper (short, worth reading).
- Alsabti, K. et al. (1998). *An Efficient K-Means Clustering Algorithm*. — for the triangle-inequality pruning idea.
- Ding, Y. et al. (2015). *Yinyang K-Means: A Drop-In Enhancement for a Fast and Accurate K-Means Algorithm*. ICML 2015. https://proceedings.mlr.press/v37/ding15.pdf
- NVIDIA RAPIDS cuML K-Means source (reference): https://github.com/rapidsai/cuml/tree/branch-25.02/cpp/src/kmeans

---

B3 · Transformer Attention Mechanism

Problem: Implement scaled dot-product attention from scratch and optimize it for long sequence lengths.

What it is: The attention mechanism is the computational core of every transformer model (BERT, GPT, ViT). For a sequence of length N, it computes an N×N attention score matrix, applies softmax, and uses the result to weight a value matrix. The memory footprint is $O(N^2)$ and the compute is $O(N^2 \cdot D)$. For long sequences (N > 512), this becomes the bottleneck. This topic gives students experience with one of the most important and actively studied GPU kernels in modern AI — FlashAttention was published in 2022 and sparked an entire line of follow-up work, all of which is about tiling this exact computation to fit in shared memory.

What you implement:

1. **CPU pipeline:** A complete transformer inference pass (embedding → attention → FFN → output) in NumPy or PyTorch CPU. Profile with `torch.profiler` to confirm attention dominates for sequences N > 256.
2. **Bottleneck:** Scaled dot-product attention — the QK^T matmul, softmax, and weighted sum. Embedding, FFN, and layer norm stay in PyTorch.
3. **GPU V1:** Naive three-kernel implementation — QK^T matmul, then softmax, then weighted sum; three separate Numba kernels writing to global memory between steps.
4. **GPU V2:** Tiled QK^T with shared memory; online softmax using the log-sum-exp trick to avoid a second global memory pass.
5. **GPU V3:** FlashAttention-style fused kernel — tile over Q, K, V simultaneously; never materialise the full N×N matrix in global memory.
6. **Benchmark:** Attention step memory bandwidth utilisation and TFLOP/s at sequence lengths 128, 512, 2048; verify output matches `torch.nn.functional.scaled_dot_product_attention` within 1e-4.

Performance target: GPU V3 kernel within 2× of PyTorch's `scaled_dot_product_attention` at sequence length 512. Target 40–120× over CPU attention step.

Dataset: No special dataset. Use synthetic Q, K, V tensors: `torch.randn(batch, heads, seq_len, head_dim)`.

Reading list:

- Vaswani, A. et al. (2017). *Attention Is All You Need*. NeurIPS 2017. — read Section 3.2 (the attention formula). https://arxiv.org/abs/1706.03762
- Dao, T. et al. (2022). *FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness*. NeurIPS 2022. — this is your GPU V3 target. Read Sections 1–3. https://arxiv.org/abs/2205.14135
- Milakov, M. & Gimelshein, N. (2018). *Online normalizer calculation for softmax*. — the online softmax algorithm you need for GPU V3. https://arxiv.org/abs/1805.02867

- The FlashAttention annotated implementation: https://github.com/Dao-AILab/flash-attention

---

B4 · BM25 Document Retrieval

Problem: Accelerate full-corpus BM25 scoring for information retrieval — find the top-K most relevant documents for a batch of queries.

What it is: BM25 is the standard ranking function used by search engines (Elasticsearch, Solr, Lucene all default to BM25). For each query term, it looks up a posting list (the list of documents containing that term), computes a term-frequency and inverse-document-frequency score, and accumulates scores across query terms to produce a final ranking. On CPU, this is done serially per query. On GPU, the challenge is irregular data access (posting lists have different lengths), load imbalance between terms, and the need for a parallel top-K selection. This topic is more unusual and gives students experience with GPU-accelerated search — a real production workload.

What you implement:

1. **CPU pipeline:** Full retrieval pipeline — index construction + query scoring — using the `rank_bm25` library for verification; your own NumPy implementation for timing. Profile to confirm posting list traversal and score accumulation dominate, not index loading.
2. **Bottleneck:** Per-query posting list scoring and score accumulation across documents. Index construction stays on CPU (done once offline).
3. **GPU V1:** Parallel posting list scoring — one thread block per query term; atomic accumulation of document scores.
4. **GPU V2:** Warp-level reduction for posting list traversal; shared memory score buffer to reduce atomic contention.
5. **GPU V3:** Batched queries — process 32 queries simultaneously with shared inverted index segments in shared memory.
6. **GPU V4 (optional):** Parallel top-K selection using CUB `DeviceRadixSort` for the final ranking step.
7. **Benchmark:** Queries per second for corpus size 100K and 1M documents; verify top-10 retrieved documents match CPU reference exactly.

Performance target: Score a corpus of 500K documents for a batch of 32 queries in under 100 ms. Target 15–50× speedup over the CPU scoring step.

Dataset:

- MS MARCO passage ranking dataset: https://microsoft.github.io/msmarco/
- Wikipedia abstract dump (smaller): https://dumps.wikimedia.org/

Reading list:

- Robertson, S. & Zaragoza, H. (2009). *The Probabilistic Relevance Framework: BM25 and Beyond*. Foundations and Trends in IR. — the definitive BM25 reference. https://www.staff.city.ac.uk/~sbrp622/papers/foundations_bm25_review.pdf
- Crane, M. et al. (2017). *Faster and Smaller Inverted Indices with Threading*. SIGIR 2017. — discusses parallel scoring strategies.
- NVIDIA CUB library (for parallel sort and reduction): https://nvlabs.github.io/cub/
- Pibiri, G. & Trani, R. (2021). *Techniques for Inverted Index Compression*. ACM CSUR.

Track C — Graph and Sparse

Graph and sparse workloads are structurally different from the previous tracks. Access patterns are irregular, data dependencies are complex, and load balancing across threads is non-trivial. These topics are harder but teach skills that do not appear naturally in dense or image-based problems.

C1 · Sparse Matrix–Vector Multiplication (SpMV)

Problem: Accelerate SpMV — the multiplication of a large sparse matrix by a dense vector — across multiple storage formats.

What it is: SpMV is a foundational kernel in scientific computing, graph analytics, and iterative solvers. It is deceptively simple (y = Ax) but hard to make fast on GPU because the matrix has irregular structure: some rows have many non-zeros, others have few. This causes severe load imbalance between thread blocks. Different sparse formats (CSR, COO, ELL, HYB) make different trade-offs between memory usage and GPU efficiency. Students implementing this topic will gain deep insight into how irregular data structures interact with GPU architecture.

What you implement:

1. **CPU pipeline:** Full iterative solver (e.g., conjugate gradient) or graph algorithm that uses SpMV as its inner loop, implemented with SciPy sparse matrices for verification and your own CSR NumPy loop for timing. Profile to confirm SpMV dominates — it will for any problem with > ~10 iterations.
2. **Bottleneck:** The SpMV kernel itself. The outer solver loop, convergence check, and result post-processing stay in Python/NumPy.
3. **GPU V1:** CSR format — one thread per row. Simple, but heavily unbalanced for skewed degree distributions.
4. **GPU V2:** CSR format — one warp per row; better load balance for rows with many non-zeros.
5. **GPU V3:** ELL format — pad all rows to the same length; enables fully coalesced memory access across the warp.

6. **GPU V4:** HYB format — ELL for the regular part + COO for outlier rows with very high degree.
7. **Benchmark:** GFLOP/s and effective memory bandwidth for each format, across 5 matrices from SuiteSparse with different sparsity patterns.

Performance target: Match cuSPARSE within 2× on at least 3 of the 5 benchmark matrices. Target 10–40× over CPU SpMV.

Dataset: SuiteSparse Matrix Collection — download any 5 matrices from different domains.

- https://sparse.tamu.edu/
- Recommended: `cage14` (biochemistry), `wiki-Talk` (social network), `parabolic_fem` (FEM), `road_usa` (road network), `TSOPF_RS_b300_c3` (power grid)

Reading list:

- Bell, N. & Garland, M. (2009). *Implementing Sparse Matrix-Vector Multiplication on Throughput-Oriented Processors*. SC 2009. — the definitive GPU SpMV reference. https://dl.acm.org/doi/10.1145/1654059.1654078
- Greathouse, J. & Daga, M. (2014). *Efficient Sparse Matrix-Vector Multiplication on GPUs using the CSR Storage Format*. SC 2014.
- NVIDIA cuSPARSE documentation: https://docs.nvidia.com/cuda/cusparse/
- Williams, S. et al. (2009). *Roofline: An Insightful Visual Performance Model for Floating Point Programs and Multicore Architectures*. — the roofline model you will use to analyze your results.

C2 · Parallel Breadth-First Search (BFS)

Problem: Implement GPU-accelerated BFS on large graphs and study the load-imbalance problem inherent to graph traversal.

What it is: BFS visits every node in a graph level by level, starting from a source. On CPU, BFS is a sequential queue-based algorithm. On GPU, the challenge is that graphs are irregular: some nodes have 1 neighbor, others have 100,000. This makes it impossible to assign equal work to each thread. Parallel BFS requires a frontier-based approach: expand all nodes in the current frontier simultaneously, then advance to the next level. The key algorithmic insight is that this is equivalent to a sparse matrix-vector multiply on a Boolean vector — connecting SpMV to graph traversal. Students will implement multiple frontiers management strategies and measure their effect on load balance.

What you implement:

1. **CPU pipeline:** Full BFS on a loaded graph using Python `collections.deque`; verify with NetworkX. Profile to confirm frontier expansion dominates for large graphs with many edges.
2. **Bottleneck:** Frontier expansion — visiting all neighbours of all nodes in the current frontier simultaneously. Graph loading and result extraction stay in Python.
3. **GPU V1:** Level-synchronous BFS — one thread per edge in the frontier; naive, produces correct results but heavily load-imbalanced for power-law graphs.
4. **GPU V2:** Work-efficient frontier expansion — use CUB prefix scan to evenly distribute edge work across threads regardless of node degree.
5. **GPU V3:** Direction-optimizing BFS (Beamer et al. 2012) — dynamically switch between top-down (push) and bottom-up (pull) expansion based on frontier density.
6. **Benchmark:** Traversed edges per second (MTEPS) and total traversal time for 5 graphs of different structure and degree distribution.

Performance target: Achieve at least 500 MTEPS on the `road_usa` graph. Target 20–80× over CPU BFS.

Dataset: SNAP graph dataset collection.

- https://snap.stanford.edu/data/
- Recommended: `roadNet-CA` (road network), `com-Youtube` (social), `wiki-Talk` (communication), `amazon0601` (co-purchase), `soc-LiveJournal1` (social)

Reading list:

- Beamer, S. et al. (2012). *Direction-Optimizing Breadth-First Search*. SC 2012. — the key algorithmic paper for GPU V3. https://parlab.eecs.berkeley.edu/sites/all/parlab/files/SC12-BFS.pdf
- Merrill, D. et al. (2012). *Scalable GPU Graph Traversal*. PPoPP 2012. — the production GPU BFS implementation. https://research.nvidia.com/publication/2012-02_scalable-gpu-graph-traversal
- NVIDIA Gunrock graph library (reference): https://github.com/gunrock/gunrock
- CUB library for parallel scan: https://nvlabs.github.io/cub/

C3 · PageRank

Problem: Implement iterative PageRank on large graphs and optimize the sparse matrix operations at its core.

What it is: PageRank computes the importance of each node in a graph based on the importance of its neighbors — the algorithm that made Google. It iterates until convergence: at each step, every node's rank is the weighted sum of its neighbors' ranks. This is equivalent to repeated sparse matrix-vector multiplication: r_new = αAr + (1-α)e, where A is the column-normalized adjacency matrix. The computational challenge on GPU is identical to SpMV — irregular

access, load imbalance — plus a convergence check requiring a global reduction after each iteration. This topic is ideal for students who completed the SpMV exercises in the course and want to apply that knowledge to a real algorithm.

What you implement:

1. **CPU pipeline:** Full PageRank power iteration using SciPy sparse matrices for verification; your own CSR implementation for timing. Profile to confirm the SpMV step dominates — the convergence check and damping update are negligible.
2. **Bottleneck:** The sparse matrix-vector multiply `r_new = αAr`. The damping factor application and convergence check can stay in NumPy or be fused as a secondary optimisation.
3. **GPU V1:** CSR SpMV kernel for the matrix-vector product; separate kernel for the damping factor update.
4. **GPU V2:** Fused kernel combining SpMV + damping factor application + L1-norm convergence reduction in a single pass.
5. **GPU V3:** Experiment with push-based (scatter) vs pull-based (gather) PageRank; use warp-level shuffle instructions for the convergence reduction; compare performance on graphs with different degree distributions.
6. **Benchmark:** Iterations per second and convergence time on 5 graphs; verify final rank vector matches SciPy within 1e-6 relative error.

Performance target: Converge PageRank (tolerance 1e-6) on `com-Youtube` in under 5 seconds. Target 15–60× over CPU SpMV step.

Dataset: Same SNAP collection as C2.

- https://snap.stanford.edu/data/
- Recommended same set: `roadNet-CA`, `com-Youtube`, `wiki-Talk`, `amazon0601`, `soc-LiveJournal1`

Reading list:

- Page, L. et al. (1999). *The PageRank Citation Ranking: Bringing Order to the Web*. Stanford Technical Report. https://infolab.stanford.edu/pub/papers/google.pdf
- Malewicz, G. et al. (2010). *Pregel: A System for Large-Scale Graph Processing*. SIGMOD 2010. — context for graph computation models.
- McLaughlin, A. & Bader, D. (2014). *Scalable and High Performance PageRank on the GPU*. SC 2014.
- GraphBLAS specification (for the SpMV-as-graph-primitive perspective): https://graphblas.org/

Track D — Deep Learning

Deep learning topics in this track follow a different philosophy from the other tracks. **You do not need to implement the full pipeline on GPU.** Instead, you start with a working PyTorch model, profile it, identify one computationally intensive bottleneck, and replace only that part with a custom CUDA kernel written in Numba or CuPy. The rest of the model stays in PyTorch. The goal is to understand what is happening inside the framework — and beat it on a specific operation.

Each topic therefore has two correctness checks: the custom kernel must produce the same output as the PyTorch reference within floating-point tolerance, and the end-to-end model must produce the same predictions before and after the substitution.

D1 · Custom Attention Kernel for Vision Transformer (ViT)

Problem: Replace the scaled dot-product attention in a pretrained ViT with a hand-written CUDA kernel and measure the speedup on the attention step alone.

What it is: Vision Transformers apply multi-head self-attention to sequences of image patches. For an input image split into N patches, attention computes an N×N score matrix per head — an $O(N^2)$ operation that dominates inference time for large images or long sequences. PyTorch's built-in `scaled_dot_product_attention` is already well-optimized, so this topic is genuinely hard: your kernel must compete with cuDNN. The learning value is understanding exactly what FlashAttention does and why the naive tiled implementation falls short. Students who complete this topic will have a deep understanding of the single most important kernel in modern AI.

What you implement:

1. CPU baseline: ViT forward pass in PyTorch (CPU), record attention step latency using `torch.profiler`
2. GPU reference: ViT forward pass in PyTorch (GPU) with `torch.nn.functional.scaled_dot_product_attention` — this is your comparison target
3. GPU V1: Naive custom kernel — compute QK^T, apply softmax, multiply by V; three separate Numba kernels
4. GPU V2: Tiled QK^T matmul with shared memory; online softmax to avoid materializing the full score matrix
5. GPU V3: FlashAttention-style fused kernel — tile over Q, K, V simultaneously; never write the N×N matrix to global memory
6. Integration: swap PyTorch attention with your kernel; verify model output is identical end-to-end
7. Benchmark: attention step latency vs `scaled_dot_product_attention` at sequence lengths 196 (ViT-S/16 on 224px), 784 (448px input)

Performance target: GPU V3 kernel within 2× of PyTorch's `scaled_dot_product_attention` at sequence length 196. Target 30–80× over CPU attention baseline.

Important constraint: Only the attention forward pass is replaced. Patch embedding, layer norm, FFN, and classifier head remain as PyTorch modules. Do not rewrite the full model.

Dataset: ImageNet validation set (50K images) for end-to-end accuracy check; synthetic tensors for kernel microbenchmarks.

- Pretrained ViT weights: `torchvision.models.vit_b_16(pretrained=True)` or `timm` library
- ImageNet validation: https://www.image-net.org/download.php (requires registration)
- Or use CIFAR-10 with a smaller ViT for quicker iteration

Reading list:

- Dosovitskiy, A. et al. (2020). *An Image is Worth 16×16 Words: Transformers for Image Recognition at Scale*. ICLR 2021. — the ViT paper. Read Sections 3.1–3.2 for the architecture. https://arxiv.org/abs/2010.11929
- Dao, T. et al. (2022). *FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness*. NeurIPS 2022. — your V3 target. Read Sections 2–3. https://arxiv.org/abs/2205.14135
- Milakov, M. & Gimelshein, N. (2018). *Online normalizer calculation for softmax*. — required reading for the numerically stable online softmax. https://arxiv.org/abs/1805.02867
- PyTorch profiler tutorial: https://pytorch.org/tutorials/recipes/recipes/profiler_recipe.html
- `timm` library (pretrained ViT models): https://github.com/huggingface/pytorch-image-models

---

D2 · Parallel Beam Search Decoding

Problem: Replace the CPU-bound beam search loop in a pretrained sequence-to-sequence model with a parallel GPU implementation.

What it is: Beam search is the standard decoding algorithm for text generation models (translation, summarization, captioning). It maintains B candidate sequences (the "beam") at each step, expands each by the full vocabulary V, scores all B×V candidates, and keeps the top B. On CPU, this loop is sequential — one step at a time, one hypothesis at a time. The model forward pass is already on GPU, but the hypothesis management (scoring, sorting, pruning) bounces back to CPU, creating a pipeline stall at every decoding step. This topic targets that stall: keep the entire beam search on GPU. The key operations — top-K selection across B×V candidates, hypothesis concatenation, score normalization — are all parallelizable and can be fused into a single kernel per decoding step.

What you implement:

1. CPU baseline: Hugging Face `model.generate()` with `num_beams=5` — this is the reference for correctness and the timing baseline
2. GPU reference: same but with `device="cuda"` — measure where time is spent using `torch.profiler`
3. GPU V1: Custom parallel top-K kernel over the B×V score matrix (one warp per beam hypothesis)
4. GPU V2: Batched hypothesis expansion — process all B hypotheses simultaneously; fuse score computation with length penalty
5. GPU V3: Fully on-GPU beam state management — hypothesis tokens, scores, and termination flags in GPU memory; eliminate all CPU↔GPU transfers inside the decode loop
6. Benchmark: tokens per second and latency per sequence for beam width B ∈ {1, 4, 8, 16}

Performance target: Generate a 100-token sequence with B=8 at least 5× faster than the CPU-baseline `generate()` call. Target 10–30× over CPU.

Important constraint: The transformer model forward pass (embedding, attention, FFN) stays in PyTorch. Only the beam management logic is replaced.

Dataset: WMT14 English→German translation dataset (newstest2014, 3,003 sentences).

- Hugging Face datasets: `datasets.load_dataset("wmt14", "de-en")`
- Pretrained model: `Helsinki-NLP/opus-mt-en-de` from Hugging Face Hub
- Or use a smaller model (T5-small, MarianMT) for faster iteration

Reading list:

- Sutskever, I. et al. (2014). *Sequence to Sequence Learning with Neural Networks*. NeurIPS 2014. — introduces the beam search decoding framework. https://arxiv.org/abs/1409.3215
- Freitag, M. & Al-Onaizan, Y. (2017). *Beam Search Strategies for Neural Machine Translation*. — practical analysis of beam width effects. https://arxiv.org/abs/1702.01806
- Shallue, C. & Senges, I. (2023). *Accelerating Beam Search for Language Models with Speculative Sampling*. — modern perspective on decoding acceleration.
- NVIDIA CUB `DeviceRadixSort` and `DeviceSelect` (for the top-K step): https://nvlabs.github.io/cub/

D3 · GPU-Accelerated Anchor Box Matching for Object Detection Training

Problem: Replace the anchor-to-ground-truth matching step in a YOLO or SSD training loop with a custom GPU kernel, eliminating a known CPU bottleneck in detection training.

What it is: Object detectors like YOLO and SSD generate thousands of anchor boxes per image. During training, each anchor must be matched to a ground truth object (or marked as

background) by computing IoU between every anchor–ground truth pair. For a single image with A=8,000 anchors and G=50 ground truth boxes, this requires computing and comparing 400,000 IoU values — done serially on CPU between every forward pass. Across a training run of 100K iterations with batch size 8, this serial matching adds hours to total training time. The entire operation is embarrassingly parallel: every anchor–ground truth pair is independent.

What you implement:

1. CPU baseline: anchor matching using NumPy IoU computation inside a training step loop; time only the matching step, not the forward/backward pass
2. GPU V1: Parallel IoU matrix kernel — one thread per (anchor, gt) pair; naive global memory writes
3. GPU V2: Tiled IoU matrix with shared memory; coalesced anchor coordinate reads
4. GPU V3: Fused matching kernel — IoU computation + argmax + threshold assignment in a single pass; eliminates the intermediate IoU matrix
5. Integration: drop the custom kernel into a real YOLOv5 or SSD training loop; verify matched anchor counts and assignments are identical to CPU reference
6. Benchmark: matching time per batch for A ∈ {2000, 8000, 32000} anchors, G ∈ {10, 50, 100} ground truth boxes

Performance target: Complete anchor matching for A=8,000, G=50, batch=8 in under 2 ms. Target 20–50× over CPU matching baseline.

Important constraint: The model forward pass, loss computation, and backpropagation stay in PyTorch. Only the matching step is replaced.

Dataset: COCO 2017 detection dataset.

- Download: https://cocodataset.org/#download
- Or use Pascal VOC 2012 (smaller): http://host.robots.ox.ac.uk/pascal/VOC/
- YOLOv5 starter code: https://github.com/ultralytics/yolov5

Reading list:

- Redmon, J. et al. (2016). *You Only Look Once: Unified, Real-Time Object Detection*. CVPR 2016. — the YOLO paper; explains anchor matching in Section 2. https://arxiv.org/abs/1506.02640
- Liu, W. et al. (2016). *SSD: Single Shot MultiBox Detector*. ECCV 2016. — explains the matching strategy in Section 2.2. https://arxiv.org/abs/1512.02325
- Rezatofighi, H. et al. (2019). *Generalized Intersection over Union: A Metric and A Loss for Bounding Box Regression*. CVPR 2019. — extends IoU to GIoU, relevant for V3. https://arxiv.org/abs/1902.09630
- `torchvision.ops.box_iou` source (CPU reference implementation): https://github.com/pytorch/vision/blob/main/torchvision/ops/boxes.py

---

D4 · Custom Depthwise Separable Convolution for MobileNet Inference

Problem: Replace the depthwise convolution layers in MobileNet with a hand-tuned CUDA kernel that outperforms cuDNN's generic GEMM path on this specific operation.

What it is: MobileNet replaces standard convolutions with depthwise separable convolutions: a depthwise conv (one filter per input channel, no cross-channel mixing) followed by a 1×1 pointwise conv. The depthwise step has very low arithmetic intensity — each output element requires only K×K multiply-adds where K is the filter size (typically 3). cuDNN handles this via its general im2col+GEMM path, which is optimized for high-arithmetic-intensity operations and wastes most of its compute on the low-intensity depthwise case. A custom kernel that avoids the im2col overhead and uses registers and shared memory aggressively can win here. This is a practical topic: depthwise conv is the bottleneck in every MobileNet-family model deployed on edge devices.

What you implement:

1. CPU baseline: MobileNetV2 inference using PyTorch on CPU; profile with `torch.profiler` to confirm depthwise conv is the bottleneck
2. GPU reference: MobileNetV2 inference on GPU with PyTorch/cuDNN — this is your comparison target
3. GPU V1: Naive Numba depthwise conv kernel — one thread per output element, direct filter application
4. GPU V2: Shared memory input tile + register-cached filter weights; minimize redundant global reads across adjacent output elements
5. GPU V3: Warp-level tiling — assign one warp to a row of output elements; exploit spatial locality across the warp with shuffle instructions
6. Integration: swap PyTorch depthwise conv layers with your kernel; verify model classification accuracy is unchanged end-to-end
7. Benchmark: kernel latency vs `torch.nn.Conv2d(groups=C)` for input sizes (1, C, H, W) with C ∈ {32, 64, 128}, H=W ∈ {56, 28, 14}

Performance target: Depthwise conv kernel within 1.5× of PyTorch's `torch.nn.Conv2d(groups=C)` for C=64, H=W=28. Target 40–100× over CPU depthwise conv baseline.

Important constraint: Only the depthwise convolution layers are replaced. Pointwise (1×1) convolutions, batch norm, and activations remain as PyTorch modules.

Dataset: ImageNet validation set for end-to-end accuracy verification; synthetic tensors for kernel microbenchmarks.

- Pretrained MobileNetV2: `torchvision.models.mobilenet_v2(pretrained=True)`
- Or use MobileNetV3: `torchvision.models.mobilenet_v3_small(pretrained=True)`

Reading list:

- Howard, A. et al. (2017). *MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications*. — introduces depthwise separable convolution. https://arxiv.org/abs/1704.04861
- Sandler, M. et al. (2018). *MobileNetV2: Inverted Residuals and Linear Bottlenecks*. CVPR 2018. https://arxiv.org/abs/1801.04381
- Lavin, A. & Gray, S. (2016). *Fast Algorithms for Convolutional Neural Networks*. CVPR 2016. — Winograd algorithm relevant for 3×3 kernels. https://arxiv.org/abs/1509.09308
- NVIDIA Nsight Compute documentation (for roofline analysis of your kernel): https://docs.nvidia.com/nsight-compute/

D5 · Accelerated Non-Local Means Denoising as Neural Network Post-Processing

Problem: Replace the Non-Local Means (NLM) post-processing step in a denoising pipeline with a parallel GPU kernel, accelerating the $O(N^2)$ patch search that makes NLM unusable at high resolution on CPU.

What it is: Non-Local Means denoising estimates each pixel's clean value as a weighted average of all similar patches in the image. "Similarity" is measured as the L2 distance between two image patches centered at each pair of pixels — making the naive algorithm $O(N^2)$ in the number of pixels, which is completely impractical on CPU for images larger than ~256×256. In a modern denoising pipeline, a neural network (DnCNN, FFDNet) removes most of the noise, and NLM is applied as a refinement step to recover fine texture. The GPU implementation parallelizes the patch comparison: all pixel pairs can be compared simultaneously, reducing the complexity from sequential $O(N^2)$ to parallel $O(N^2/P)$ where P is the number of threads. This topic teaches students to accelerate a fundamentally quadratic algorithm through parallelism rather than algorithmic reduction.

What you implement:

1. CPU baseline: NLM post-processing using `skimage.restoration.denoise_nl_means`; time the NLM step alone, not the neural network inference
2. GPU V1: Naive parallel patch distance kernel — one thread per pixel pair; compute L2 patch distance and weighted contribution
3. GPU V2: Shared memory patch cache — load the reference patch into shared memory once and reuse it for all comparisons within a thread block
4. GPU V3: Search window restriction + integral image acceleration — limit comparison to a local search window (e.g., 21×21); use a precomputed integral image for fast patch distance computation
5. Integration: connect GPU NLM to a pretrained DnCNN or FFDNet in PyTorch; verify PSNR of the combined pipeline matches the CPU reference
6. Benchmark: NLM step latency for image sizes 256×256, 512×512, 1024×1024; PSNR vs CPU NLM baseline

Performance target: Process a 512×512 image through NLM in under 100 ms. Target 50–200× over CPU NLM baseline.

Important constraint: The neural network denoising step stays in PyTorch. Only the NLM post-processing kernel is implemented in CUDA.

Dataset: BSD68 denoising benchmark (68 grayscale images, standard in denoising literature).

- BSD68 download: https://github.com/clausmichele/CBSD68-dataset
- Pretrained DnCNN weights: https://github.com/cszn/DnCNN
- Or use any natural images; NLM does not require labeled data

Reading list:

- Buades, A. et al. (2005). *A Non-Local Algorithm for Image Denoising*. CVPR 2005. — the original NLM paper, only 6 pages. https://www.iro.umontreal.ca/~mignotte/IFT6150/Articles/Buades-NLM.pdf
- Zhang, K. et al. (2017). *Beyond a Gaussian Denoiser: Residual Learning of Deep CNN for Image Denoising*. IEEE TIP. — the DnCNN paper; explains the neural component. https://arxiv.org/abs/1608.03981
- Darbon, J. et al. (2008). *Fast Nonlocal Filtering Applied to Electron Cryomicroscopy*. ISBI 2008. — introduces the integral image trick for fast NLM (your GPU V3 target).
- scikit-image NLM source (CPU reference): https://github.com/scikit-image/scikit-image/blob/v0.22.0/skimage/restoration/_nl_means_denoising.pyx

---

Custom Topic

Choosing your own topic is encouraged. The catalog above exists to give you concrete examples of well-scoped projects — not to limit what you can work on. If you have a problem from your research, an internship, or your own interests that involves a computationally expensive step that could benefit from a custom CUDA kernel, that is an excellent project. Students who choose their own topics consistently produce the most interesting final presentations.

Custom topics require **instructor sign-off by the end of the Week 6 session**. Bring your one-page pitch to the session; the instructor will review it during the team formation period (1:00–1:20). A topic that is not signed off by that deadline defaults to a catalog topic — not because custom topics are discouraged, but because you need the full Part 2 timeline to execute well.

What makes a valid custom topic

A valid custom topic must satisfy all four criteria:

1. Clear GPU bottleneck. The core computation must be parallel in a way that maps naturally onto GPU threads — not just "this is slow on CPU." You must be able to name the specific operation that will be parallelized (e.g., "matrix-vector multiply," "per-pixel filter," "all-pairs distance computation") and explain why it is independent across threads.

2. Measurable performance target. You must be able to state a single concrete number: "X× speedup over single-threaded CPU baseline on input of size N." If you cannot name a number before you start, you cannot evaluate your own work.

3. At least three distinct optimization levels. The project must support a natural progression: naive GPU port → memory optimization → compute optimization. A problem that can only be optimized in one way (e.g., just replacing a loop with a CuPy call) is not suitable.

4. Accessible dataset and ground truth. The dataset must be publicly downloadable. There must be a way to verify GPU output against a trusted CPU reference (analytical answer, reference library output, or labeled ground truth).

Topics that are NOT acceptable

- Topics that are fully solvable by swapping NumPy for CuPy with no kernel writing (e.g., "use CuPy for matrix inversion")
- Topics where the GPU is not the bottleneck (e.g., I/O-bound pipelines)
- Topics requiring proprietary datasets the instructor cannot access for grading
- Topics with no verifiable ground truth
- Topics that are pure simulations with no real-world relevance to AI, ML, or CV

What You Need to Submit

The following items are due **before the start of the Week 5 session**. Late submissions receive a 20% deduction per day.

1. Project Proposal (required, graded on completeness)

Submit to the course LMS as a PDF or Markdown file. Use the template in `project_proposal_template.md`. Required content:

- Title, team members, topic name and track
- Git repository URL (must be accessible to instructor)
- Problem statement with dataset details
- Measured CPU baseline timing (run your baseline, paste the output)
- Performance target — one sentence with a concrete number
- Optimization plan — the table from the proposal template
- Risk analysis — at least 3 risks with mitigations
- Division of work across team members

Graded on completeness (all sections present) and specificity (performance target is a number, not a vague goal). Quality of writing is not assessed at this stage.

2. Git Repository (required, graded pass/fail)

Create a public or instructor-accessible repository before the end of the Week 6 session. It must contain:

```
your_project/
├── README.md           ← project title, team, one-line description
├── requirements.txt    ← all Python dependencies with versions
├── src/
│   └── cpu_baseline.py ← must run: python src/cpu_baseline.py
└── tests/
    └── test_correctness.py ← must pass: pytest tests/
```

The `cpu_baseline.py` must run end-to-end on any machine with the listed dependencies (no GPU required). If it fails to run, the Week 6 deliverable receives a zero.